



Gobierno de
México

Ciencia y Tecnología
Secretaría de Ciencia, Humanidades, Tecnología e Innovación

UNRC
Universidad Nacional Rosario Castellanos



ESTRUCTURA DE DATOS



2025
Año de
La Mujer
Indígena

Avenida 506 Col. San Juan de Aragón II Sección, Gustavo A. Madero C.P. 07969, Ciudad de México www.rcastellanos.cdmx.gob.mx



Descripción del programa

Objetivo general

El programa gestiona los turnos de atención del sistema PAD-Bienestar. La idea es simple: los usuarios llegan, se registran en una fila virtual y se atienden en el mismo orden en que llegaron. Para eso se usa una cola dinámica implementada en lenguaje C, donde cada turno es un nodo independiente que se crea y destruye en memoria durante la ejecución del programa.

El programa tiene cuatro operaciones concretas: agregar un turno al final de la fila, atender el turno del frente eliminando su nodo, mostrar todos los turnos pendientes y detectar cuando la cola está vacía para no ejecutar operaciones sin sentido.

Estructura de datos utilizada

La estructura de datos es una cola dinámica enlazada. A diferencia de un arreglo, no tiene un tamaño definido desde el inicio; crece o se reduce conforme se agregan o atienden turnos. Cada elemento de la cola es un nodo del siguiente tipo:

```
struct Turno {  
    int numero;  
    struct Turno *siguiente;  
};
```

El campo número guarda el identificador del turno. El campo siguiente es un apuntador al próximo nodo, lo que permite encadenar todos los turnos sin necesitar un arreglo contiguo en memoria. Dos apuntadores globales, frente y final, marcan el inicio y el final de la cola



Relación entre la estructura y el problema

En una sala de espera real, la primera persona en llegar es la primera en ser atendida. Eso es exactamente lo que hace una cola con el principio FIFO (First In, First Out). El apuntador frente apunta siempre al turno más antiguo, el que se atiende primero. El apuntador final apunta al turno más reciente. Cuando se agrega un turno, se enlaza al final; cuando se atiende uno, se elimina del frente. Ningún turno puede saltarse la fila.

Lógica del sistema

Qué pasa en memoria cuando se agrega un turno (malloc)

Cuando el usuario elige agregar un turno, el programa llama a malloc con el tamaño exacto de un nodo struct Turno. Eso reserva un bloque de memoria en el heap, que es la zona donde el programa administra asignaciones en tiempo de ejecución. Si la reserva falla (por ejemplo, si no hay memoria disponible), malloc regresa NULL y el programa lo detecta antes de continuar.

Si la reserva es exitosa, se guarda el número de turno en el campo numero y se pone siguiente en NULL porque el nodo nuevo siempre va al final. Después se distinguen dos casos: si la cola estaba vacía, el nodo nuevo es al mismo tiempo el frente y el final; si ya había nodos, el campo siguiente del nodo anterior se enlaza al nuevo y el apuntador final se mueve hacia él.

Qué pasa en memoria cuando se atiende un turno (free)

Atender un turno consiste en eliminar el nodo del frente. Primero se guarda su dirección en un apuntador temporal para no perderla. Luego el apuntador frente avanza al siguiente nodo. Si después del avance el frente queda en NULL, significa que la cola quedó vacía, así que el apuntador final también se pone en NULL para mantener consistencia.

Por último, se llama a free con el apuntador temporal. Eso le devuelve el bloque de memoria al sistema operativo. Sin ese paso, el nodo quedaría inutilizable, pero ocupando espacio, que es lo que se conoce como fuga de memoria (memory leak). El programa también incluye la función liberar_cola que recorre y libera todos los nodos que queden al salir.



Cómo se recorre la cola para mostrar los turnos

La función `mostrar_cola` usa un apuntador auxiliar llamado `actual` que empieza apuntando al frente. En cada iteración imprime el número del nodo `actual` y luego mueve el apuntador al siguiente nodo. El ciclo termina cuando `actual` llega a `NULL`, que es la marca del final de la cola. En ningún momento se modifica el apuntador frente original, así que la cola queda intacta.

Cómo se mantiene el orden FIFO

El orden de atención lo garantizan los dos apuntadores. Los turnos se insertan siempre por final y se eliminan siempre por frente. No hay ninguna operación que permita insertar en medio de la cola ni saltarse el frente. Si se agregan los turnos 101, 102 y 103 en ese orden, el frente apuntará a 101, y ese será el primero en atenderse. Cuando se atiende el 101, el frente pasa al 102, y así sucesivamente.

Evidencia

Ejecución con datos correctos (agregar y mostrar turnos)

Se compiló el programa con `gcc` y se ejecutó en terminal `vscode`. Primero se registraron tres turnos (101, 102 y 103) y se verificó que la cola los muestra en el orden correcto:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

s\bin\WindowsDebugLauncher.exe' '--stdin=Micro
ednn0.0e2' '--pid=Microsoft-MIEngine-Pid-j3

=====
Sistema MARVIN - MAYRA -Turnos
=====

--- MENU PRINCIPAL ---
1. Agregar turno
2. Atender turno
3. Mostrar cola de turnos
0. Salir
Selecciona una opcion: 1
Ingresa el numero de turno: 101
Turno 101 registrado en la fila.

--- MENU PRINCIPAL ---
1. Agregar turno
2. Atender turno
3. Mostrar cola de turnos
0. Salir
Selecciona una opcion: 1
Ingresa el numero de turno: 102
Turno 102 registrado en la fila.

--- MENU PRINCIPAL ---
1. Agregar turno
2. Atender turno
3. Mostrar cola de turnos
0. Salir
Selecciona una opcion: 1
Ingresa el numero de turno: 103
Turno 103 registrado en la fila.
```





Atender turnos respetando el orden FIFO

Se atendieron los tres turnos en secuencia. El programa siempre elimina el frente de la cola, lo que confirma que el orden de atención es el mismo en que llegaron:

```
--- MENU PRINCIPAL ---  
1. Agregar turno  
2. Atender turno  
3. Mostrar cola de turnos  
0. Salir  
Selecciona una opcion: 2  
Atendiendo turno: 101
```

```
--- MENU PRINCIPAL ---  
1. Agregar turno  
2. Atender turno  
3. Mostrar cola de turnos  
0. Salir  
Selecciona una opcion: 2  
Atendiendo turno: 102
```

```
--- MENU PRINCIPAL ---  
1. Agregar turno  
2. Atender turno  
3. Mostrar cola de turnos  
0. Salir  
Selecciona una opcion: 2  
Atendiendo turno: 103
```



Cola vacía (validación del sistema)

Una vez atendidos todos los turnos, el programa detecta que la cola está vacía y lo indica antes de ejecutar cualquier operación inválida:

```
--- MENU PRINCIPAL ---  
1. Agregar turno  
2. Atender turno  
3. Mostrar cola de turnos  
0. Salir  
Selecciona una opcion: 2  
No hay turnos en espera. La cola está vacía.
```

Errores identificados y soluciones

Error identificado	Causa	Solución aplicada
Fuga de memoria al salir con turnos pendientes	La función main no liberaba los nodos restantes	Se añadió liberar_cola() antes de return 0
Comportamiento indefinido al atender cola vacía	Se accedía a frente->numero sin verificar si era NULL	Se agregó validación if (frente == NULL) al inicio
Apuntador final sin limpiar tras vaciar la cola	Solo se actualizaba frente, no final	Se añade final = NULL cuando frente queda en NULL



Conclusiones

La primera ventaja que noto al usar una cola dinámica en lugar de un arreglo es que no hay que decidir de antemano cuántos turnos va a manejar el sistema. Un arreglo de tamaño fijo obliga a escoger entre desperdiciar memoria o quedarse corto. Con la cola dinámica, cada turno ocupa exactamente lo que necesita y se libera cuando ya no sirve. En un sistema real como PAD-Bienestar, donde la carga varía a lo largo del día, eso marca una diferencia práctica.

El manejo explícito de memoria con malloc y free obliga a entender qué ocurre en el heap en cada paso del programa. No es solo un detalle técnico: saber que cada llamada a malloc reserva un bloque y que ese bloque no desaparece solo hasta que se llama a free cambia la forma de diseñar funciones. Si no se libera la memoria a tiempo, el programa sigue corriendo, pero consumiendo recursos de más, algo que en producción puede escalar hasta volver un sistema inestable.

Finalmente, la relación entre la estructura de datos y el problema que resuelve no es accidental. El principio FIFO de la cola refleja directamente la idea de equidad en una sala de espera: el primero en llegar es el primero en ser atendido. Elegir la estructura correcta hace que el código sea directo, sin trucos ni condiciones adicionales para mantener el orden. Eso es lo que distingue a una solución bien pensada de una que simplemente funciona.



Gobierno de
México

Ciencia y Tecnología
Secretaría de Ciencia, Humanidades, Tecnología e Innovación

UNRC
Universidad Nacional Rosario Castellanos



Referencias

Cormen, T. H., Leiserson, C. E., Rivest, R. L., y Stein, C. (2022). Introduction to algorithms (4.^a ed.). MIT Press.
<https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/>

The GNU Project. (2024). The GNU C Library reference manual: Memory allocation. Free Software Foundation.
https://www.gnu.org/software/libc/manual/html_node/Memory-Allocation.html

Sedgewick, R., y Wayne, K. (2011). Algorithms (4.^a ed.). Addison-Wesley.
<https://algs4.cs.princeton.edu/home/>